```
import cv2
import numpy as np
import matplotlib.pyplot as plt

print("OpenCV Version:", cv2.__version__)
print("Libraries imported successfully!")
```

```
OpenCV Version: 5.0.0
Libraries imported successfully!
```

```
width, height = 640, 360
num_frames = 50

frames = []

for i in range(num_frames):
    frame = np.zeros((height, width, 3), dtype=np.uint8)

    # Moving rectangle
    x = 30 + i * 6
    cv2.rectangle(frame, (x, 60), (x + 70, 130),
                  (0, 255, 255), -1)

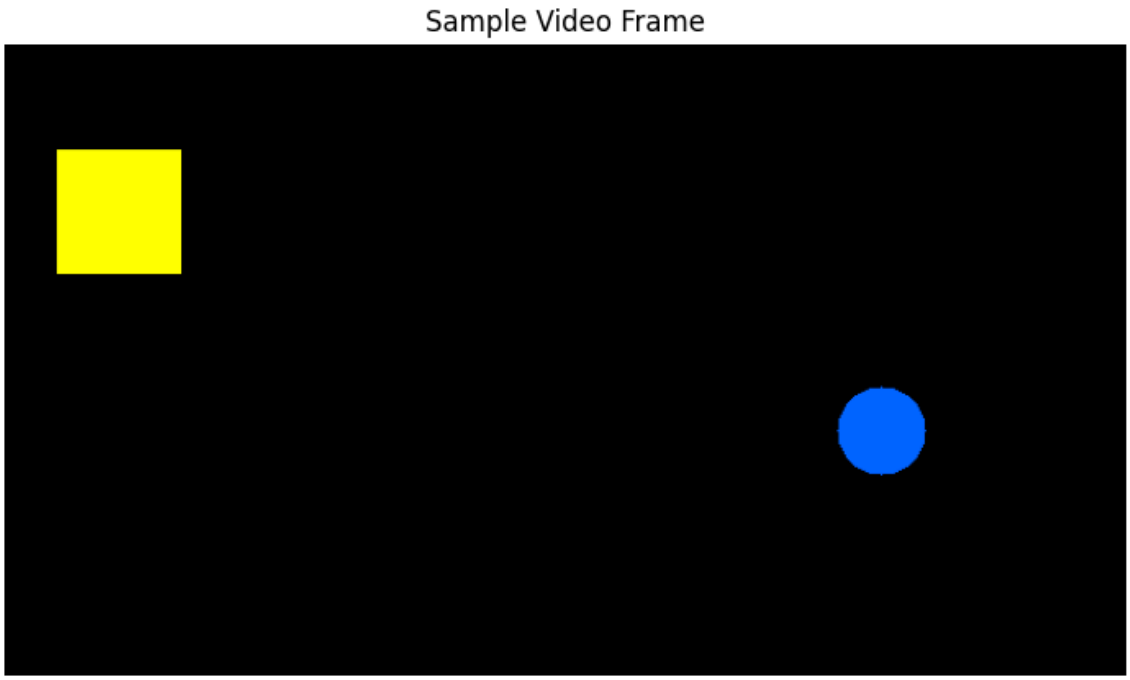
    # Moving circle
    cx = 500 - i * 4
    cy = 220 + int(30 * np.sin(i / 5))

    cv2.circle(frame, (cx, cy), 25, (255, 100, 0), -1)

    frames.append(frame)

plt.figure(figsize=(10, 5))
plt.imshow(cv2.cvtColor(frames[0], cv2.COLOR_BGR2RGB))
plt.title("Sample Video Frame")
plt.axis("off")
plt.show()

print("Sample frames created:", len(frames))
```




```
Sample frames created: 50
```

```
old_gray = cv2.cvtColor(frames[0], cv2.COLOR_BGR2GRAY)

p0 = cv2.goodFeaturesToTrack(
    old_gray,
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7,
    blockSize=7
)

if p0 is None:
    raise ValueError("No feature points detected.")

mask = np.zeros_like(frames[0])
lk_params = {
    "winSize": (15, 15),
    "maxLevel": 2,
    "criteria": (
        cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT,
        10, 0.03
    )
}

for i in range(1, len(frames)):
    frame = frames[i].copy()
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    p1, status, error = cv2.calcOpticalFlowPyrLK(
        old_gray, gray, p0, None, **lk_params
    )

    if p1 is not None and status is not None:
        good_new = p1[status.ravel() == 1]
        good_old = p0[status.ravel() == 1]

        for new, old in zip(good_new, good_old):
            a, b = np.round(new).astype(int).ravel()
            c, d = np.round(old).astype(int).ravel()

            cv2.line(mask, (a, b), (c, d), (0, 255, 0), 2)
            cv2.circle(frame, (a, b), 4, (0, 0, 255), -1)

        if len(good_new) >= 5:
            p0 = good_new.reshape(-1, 1, 2)
        else:
            p0 = cv2.goodFeaturesToTrack(
                gray, maxCorners=100, qualityLevel=0.3,
                minDistance=7, blockSize=7
            )

    if p0 is None:
        p0 = cv2.goodFeaturesToTrack(
            gray, maxCorners=100, qualityLevel=0.3,
            minDistance=7, blockSize=7
        )

    if p0 is None:
        old_gray = gray
        continue

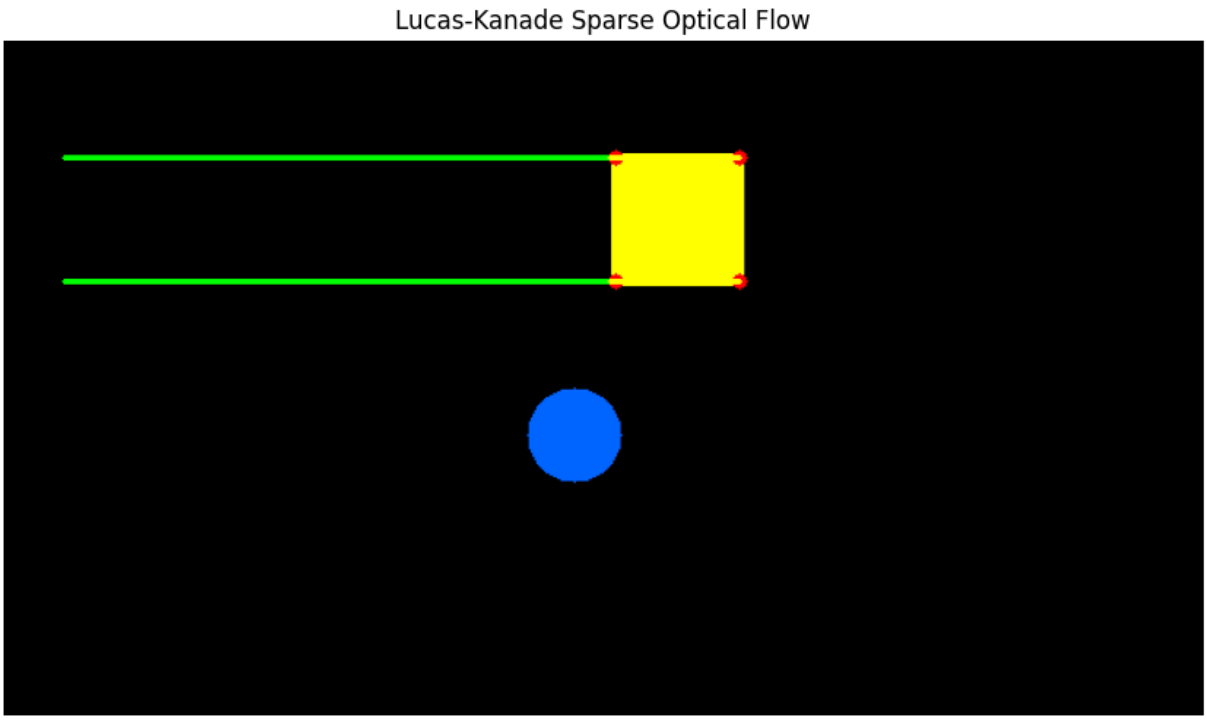
    old_gray = gray.copy()

result = cv2.add(frame, mask)

plt.figure(figsize=(12, 6))
plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
plt.title("Lucas-Kanade Sparse Optical Flow")
```

```
plt.axis("off")
plt.show()

print("Lucas-Kanade completed successfully!")
```




```
Lucas-Kanade completed successfully!
```

```
prev_gray = cv2.cvtColor(frames[0], cv2.COLOR_BGR2GRAY)

for i in range(1, len(frames)):
    next_gray = cv2.cvtColor(frames[i], cv2.COLOR_BGR2GRAY)

    flow = cv2.calcOpticalFlowFarneback(
        prev_gray,
        next_gray,
        None,
        0.5,
        3,
        15,
        3,
        5,
        1.2,
        0
    )

    magnitude, angle = cv2.cartToPolar(
        flow[..., 0], flow[..., 1]
    )

    hsv = np.zeros_like(frames[i])
    hsv[..., 1] = 255
    hsv[..., 0] = (angle * 180 / np.pi / 2).astype(np.uint8)

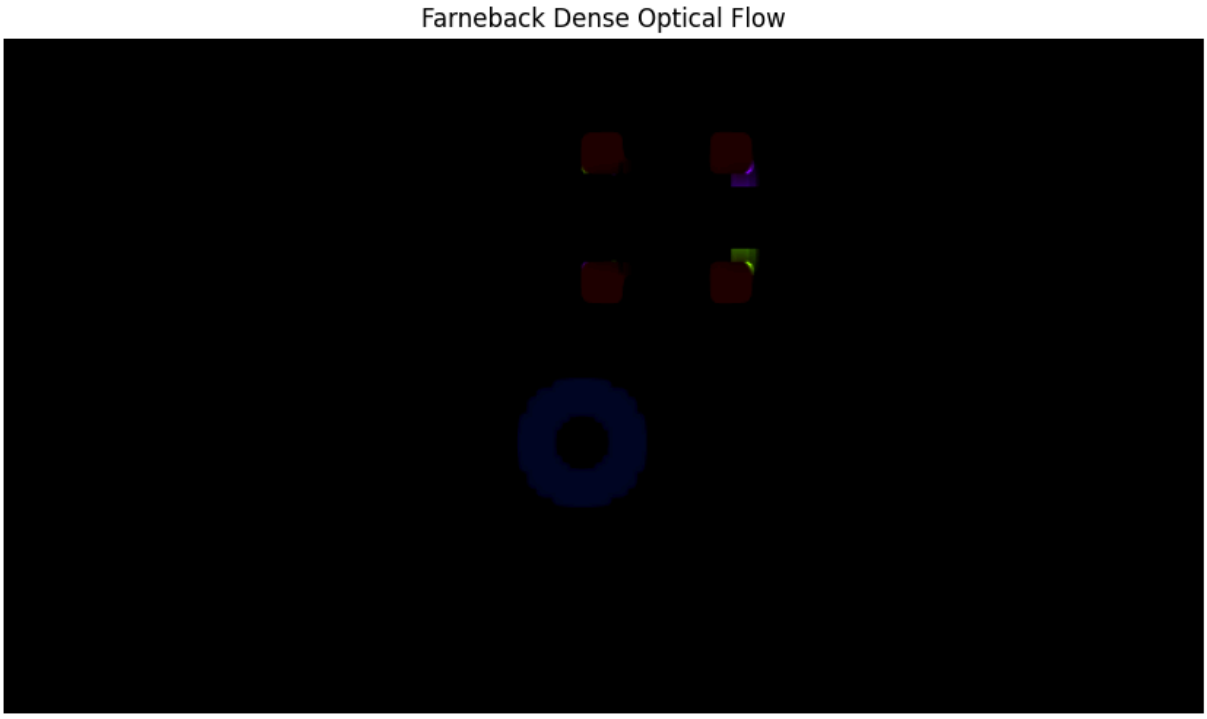
    hsv[..., 2] = cv2.normalize(
        magnitude, None, 0, 255, cv2.NORM_MINMAX
    ).astype(np.uint8)

    optical_flow_image = cv2.cvtColor(
        hsv, cv2.COLOR_HSV2RGB
    )
```

```
        prev_gray = next_gray

plt.figure(figsize=(12, 6))
plt.imshow(optical_flow_image)
plt.title("Farneback Dense Optical Flow")
plt.axis("off")
plt.show()

print("Farneback completed successfully!")
```




```
Farneback completed successfully!
```

```
print("OPTICAL FLOW COMPARISON")
print("-" * 40)
print("Lucas-Kanade: Sparse feature tracking")
print("Farneback: Dense pixel-wise motion estimation")
print("Lucas-Kanade: Useful for tracking selected points")
print("Farneback: Useful for visualizing overall scene motion")
```

```
OPTICAL FLOW COMPARISON
----------------------------------------
Lucas-Kanade: Sparse feature tracking
Farneback: Dense pixel-wise motion estimation
Lucas-Kanade: Useful for tracking selected points
Farneback: Useful for visualizing overall scene motion
```

```
answers = {
    "Q1. What is Optical Flow? How is it used in computer vision?":
    "Optical Flow estimates apparent motion between consecutive video frames. It is used in o

    "Q2. Explain the working principle of Lucas-Kanade Optical Flow.":
    "Lucas-Kanade detects feature points such as corners and tracks their displacement betwee

    "Q3. What is Dense Optical Flow? How does it differ from Sparse Optical Flow?":
    "Dense Optical Flow estimates motion for nearly every pixel, whereas Sparse Optical Flow

    "Q4. Compare Lucas-Kanade and Farneback algorithms.":
    "Lucas-Kanade is a sparse feature-tracking method and is often computationally efficient.

    "Q5. What assumptions are made while computing optical flow?":
    "Common assumptions include brightness constancy, small motion between frames and locally
```

```
    "Q6. What factors affect optical-flow accuracy?":
    "Lighting changes, motion blur, low-texture regions, noise, occlusion, fast movement and

    "Q7. Mention five real-world applications of optical flow.":
    "1. Autonomous driving  2. Video surveillance  3. Robotics  4. Gesture recognition  5. Sp

    "Q8. Why are grayscale images generally used for optical flow?":
    "Grayscale images simplify computation because they contain one intensity channel instead

    "Q9. What are the limitations of optical-flow algorithms?":
    "They may be unreliable with changing illumination, motion blur, occlusion, low-texture s

    "Q10. How does optical flow help autonomous driving, surveillance and action recognition?
    "It helps estimate vehicle and pedestrian motion, detect movement in monitored scenes and
}

for question, answer in answers.items():
    print(question)
    print(answer)
    print("-" * 80)
```

```
Q1. What is Optical Flow? How is it used in computer vision?
Optical Flow estimates apparent motion between consecutive video frames. It is used in object
--------------------------------------------------------------------------------
Q2. Explain the working principle of Lucas-Kanade Optical Flow.
Lucas-Kanade detects feature points such as corners and tracks their displacement between cons
--------------------------------------------------------------------------------
Q3. What is Dense Optical Flow? How does it differ from Sparse Optical Flow?
Dense Optical Flow estimates motion for nearly every pixel, whereas Sparse Optical Flow estima
--------------------------------------------------------------------------------
Q4. Compare Lucas-Kanade and Farneback algorithms.
Lucas-Kanade is a sparse feature-tracking method and is often computationally efficient. Farne
--------------------------------------------------------------------------------
Q5. What assumptions are made while computing optical flow?
Common assumptions include brightness constancy, small motion between frames and locally consi
--------------------------------------------------------------------------------
Q6. What factors affect optical-flow accuracy?
Lighting changes, motion blur, low-texture regions, noise, occlusion, fast movement and camera
--------------------------------------------------------------------------------
Q7. Mention five real-world applications of optical flow.
1. Autonomous driving  2. Video surveillance  3. Robotics  4. Gesture recognition  5. Sports a
--------------------------------------------------------------------------------
Q8. Why are grayscale images generally used for optical flow?
Grayscale images simplify computation because they contain one intensity channel instead of th
--------------------------------------------------------------------------------
Q9. What are the limitations of optical-flow algorithms?
They may be unreliable with changing illumination, motion blur, occlusion, low-texture surface
--------------------------------------------------------------------------------
Q10. How does optical flow help autonomous driving, surveillance and action recognition?
It helps estimate vehicle and pedestrian motion, detect movement in monitored scenes and ident
--------------------------------------------------------------------------------
```

```
print("RESULT:")
print("Lucas-Kanade Sparse Optical Flow and Farneback Dense Optical Flow")
print("were implemented using Python and OpenCV.")

print("\nCONCLUSION:")
print("Lucas-Kanade tracks selected feature points, while Farneback")
print("estimates motion across nearly every pixel. Both methods help")
print("analyze motion in video sequences.")
```

```
RESULT:
Lucas-Kanade Sparse Optical Flow and Farneback Dense Optical Flow
were implemented using Python and OpenCV.

CONCLUSION:
Lucas-Kanade tracks selected feature points, while Farneback
```

```
    estimates motion across nearly every pixel. Both methods help
    analyze motion in video sequences.
```